
PRUEBA TÉCNICA PRÁCTICA: FRONTEND ENGINEER

- **EJERCICIO 1: Componente de Búsqueda Reactiva (Tiempo: 45 min)**
- **EJERCICIO 2: Gestión de Estado Simple (Signals) (Tiempo: 30 min)**
- **EJERCICIO 3: Debugging y Performance (Tiempo: 30 min)**

EJERCICIO 1: Componente de Búsqueda Reactiva (Tiempo:45 min)

Enunciado: Crea un componente CustomerSearchComponent (Standalone) que:

1. Tenga un input de texto.
2. Haga peticiones a una API simulada (MockService.search(term)) mientras el usuario escribe.
3. Muestre los resultados en una lista .

Requerimientos RxJS:

No disparar la búsqueda hasta que el usuario deje de escribir por 400ms (Debounce).

Si el usuario cambia el texto, cancelar la petición anterior si aún está en vuelo (SwitchMap).

No buscar si el texto es igual al anterior (DistinctUntilChanged).

No buscar si el texto tiene menos de 3 caracteres (Filter).

EJERCICIO 2: Gestión de Estado Simple (Signals) (Tiempo:30 min)

Enunciado: Tienes un Carrito de Compras. Implementa un servicio CartService usando

Angular Signals (Angular 17+).

1. Debe tener una lista de items (Signal de lectura).
2. Método addItem(product) que actualice la lista y recalculé el Total automáticamente.
3. Un computed signal que exponga el totalPrice (Suma de precios) y totalCount (Items).
4. Efecto (effect) que haga un console.log cada vez que el carrito cambie.

EJERCICIO 3: Debugging y Performance (Tiempo: 30 min)

Enunciado: Revisa el siguiente fragmento de código (teórico) y encuentra 3 errores de

performance/lógica:

```
@Component({
  template: `
    <div *ngFor="let item of items">
      {{ expensiveCalculation(item) }}
    </div>
  `,
})
export class ListComponent {
  @Input() items: any[];

  expensiveCalculation(item) {
    console.log("Calculando..."); // Simula proceso pesado
    return Math.random();
  }
}
```

